

\$ ADDENDUM · V2.0 · SPECIFICATION DRIVEN
// funcionalidades_no_contempladas.md

SDD · BDD · FIREBASE

AutoDev / NodeW v2.0

Addendum de Nuevas Funcionalidades — Funcionalidades No Contempladas en v1.0

Documento técnico complementario que identifica, especifica y traza las **9 funcionalidades marcadas como (Nuevo)** en la especificación v2.0 frente al baseline v1.0. Cubre requisitos funcionales, escenarios BDD/Gherkin, impacto en la arquitectura Firebase (Firestore · Cloud Functions · Auth · Storage), matriz de trazabilidad y estimación MoSCoW para evolución v1.0 → v2.0.

LAND · 02

PROF · 02

GAM · 02

RES · 01

COM · 01

// VERSIÓN
2.0 — RC

// FECHA
Julio 2026

// BASE
Firebase Stack

// METODOLOGÍA
SDD + BDD

● READY_FOR_REVIEW // ADDENDUM_V2.0 · DOC_ID: AUTODEV.SRS.ADDENDUM.V2

// tabla_de_contenidos

Índice del Addendum

1. Resumen Ejecutivo	3
1.1 Cuadro Resumen de Funcionalidades Nuevas	3
2. Análisis Comparativo v1.0 → v2.0	5
2.1 Tabla de Brecha (Gap Analysis)	5
2.2 Lectura del Análisis	5
3. Nuevas Funcionalidades Detalladas	7
3.1 RF-LAND-01 — Landing Page con Terminal Interactiva	7
3.2 RF-LAND-02 — Contadores en Tiempo Real sin Comprometer Cuota	7
3.3 RF-PROF-02 — Vista Previa en Tiempo Real del Perfil	9
3.4 RF-PROF-05 — Estado Vacío en Mis Estadísticas	9
3.5 RF-GAM-03 — Reglas de XP y Recompensas Explícitas en el Leaderboard	10
3.6 RF-GAM-04 — Sistema de Logros (Achievements) de Primera Clase	10
3.7 RF-RES-02 — Límite Estricto de 3 Adjuntos por Recurso	12
3.8 RF-COM-03 — Estado Vacío en Sección Directos	12
4. Criterios de Aceptación y Escenarios BDD	14
4.1 Escenarios — Landing Page (RF-LAND-01, RF-LAND-02)	14
4.2 Escenarios — Perfil y Estadísticas (RF-PROF-02, RF-PROF-05)	14
4.3 Escenarios — Gamificación (RF-GAM-03, RF-GAM-04)	16
4.4 Escenarios — Recursos y Comunidad (RF-RES-02, RF-COM-03)	17
5. Impacto en la Arquitectura Firebase	18
5.1 Cambios en el Modelo de Datos Firestore	18
5.2 Cloud Functions — Nuevas y Modificadas	18
5.3 Reglas de Seguridad Firestore	19
5.4 Cloud Storage — Cambios y Límites	21
5.5 Estimación de Cuota y Costo Mensual	21
6. Matriz de Trazabilidad y Priorización MoSCoW	22
6.1 Matriz de Trazabilidad	22
6.2 Priorización MoSCoW y Plan de Sprint	22
6.3 Plan de Implementación y Definición de Hecho (DoD)	23

6.4 Riesgos Técnicos y Mitigaciones	23
---	----

1. Resumen Ejecutivo

Este addendum complementa el **Documento de Especificaciones del Sistema AutoDev / NodeW v2.0** contrastándolo contra el baseline **SRS v1.0** (fechado el 10 de Julio, 2026) y consolidando todas las funcionalidades marcadas explícitamente como **(Nuevo)** en la especificación v2.0. El objetivo es ofrecer al equipo de desarrollo, al equipo de QA y al equipo de producto una visión única, trazable y priorizada de los cambios necesarios para evolucionar la plataforma desde la versión 1.0 (Release Candidate) hasta la versión 2.0 sin perder el contexto arquitectónico de Firebase Ecosystem (Firestore, Cloud Functions, Auth, Storage, FCM y Analytics) heredado del SRS original.

Tras la lectura completa del SRS v1.0 —que contiene 75 requisitos funcionales distribuidos en 11 módulos (Auth, Onboarding y Perfil, Foro, Cursos, Recursos, Directos, Miembros, Ranking, Gamificación, Notificaciones y Administración)— se identifican **9 requisitos funcionales nuevos** en v2.0, distribuidos en 5 de los 6 módulos listados en el brief visual. Estos 9 RF se desglosan a continuación: **RF-LAND-01 y RF-LAND-02** (Landing Page con terminal interactiva y contadores pre-agregados), **RF-PROF-02 y RF-PROF-05** (vista previa en tiempo real del perfil y estado vacío en estadísticas), **RF-GAM-03 y RF-GAM-04** (reglas y recompensas explícitas del leaderboard, y sistema de logros como entidad de primera clase), **RF-RES-02** (límite estricto de 3 adjuntos por recurso), y **RF-COM-03** (estado vacío en la sección de Directos).

El documento se organiza en seis capítulos: este resumen ejecutivo, un análisis comparativo v1.0 vs v2.0, las nuevas funcionalidades detalladas (con descripción, justificación de negocio e impacto técnico), los criterios de aceptación y escenarios BDD/Gherkin, el impacto en la arquitectura Firebase, y finalmente una matriz de trazabilidad junto con la estimación MoSCoW. La metodología subyacente se mantiene fiel a la combinación **SDD + BDD** definida en v1.0, reutilizando los patrones de Cloud Functions reactivas y contadores desnormalizados ya probados en producción para no introducir paradigmas nuevos en la arquitectura.

1.1 Cuadro Resumen de Funcionalidades Nuevas

ID	Módulo	Descripción	Prioridad	Esfuerzo
RF-LAND-01	Landing	Terminal interactiva + contadores globales (devs, posts, cursos, recursos)	Must	M (5 SP)
RF-LAND-02	Landing	Contadores en tiempo real con 1 lectura pre-agregada (cuota Firestore)	Must	M (5 SP)
RF-PROF-02	Perfil	Vista previa en vivo de cambios de perfil antes de guardar	Should	S (3 SP)
RF-PROF-05	Perfil	Estado vacío en Mis Estadísticas + CTA "+ Crear Post"	Should	XS (2 SP)
RF-GAM-03	Gamificación	Reglas de XP y recompensas explícitas en UI del Leaderboard	Must	XS (2 SP)

ID	Módulo	Descripción	Prioridad	Esfuerzo
RF-GAM-04	Gamificación	Sistema de Logros (Achievements) como entidad de primera clase	Must	L (8 SP)
RF-RES-02	Recursos	Límite estricto de 3 archivos adjuntos por recurso, validación pre-publicación	Must	XS (2 SP)
RF-COM-03	Comunidad	Estado vacío ("// PRÓXIMOS") cuando no hay directos programados	Should	XS (1 SP)

Tabla 1.1 — Cuadro resumen de los 8 RF nuevos identificados en v2.0.

El conteo total es de **8 RF nuevos** (no 9 como se mencionó preliminarmente; el análisis comparativo detallado confirma 8 tras verificar que algunas funciones como el reset semanal del leaderboard ya existían en v1.0 como RF-056 y RF-057). El esfuerzo total estimado asciende a aproximadamente **28 story points**, distribuidos en 4 Must, 3 Should y 1 implícito Could si se incluye la mejora visual del leaderboard ya cubierta por RF-GAM-03. Este volumen de trabajo es compatible con un sprint de 2 semanas para un equipo de 3 desarrolladores full-stack más 1 ingeniero Firebase dedicado a las Cloud Functions y reglas de seguridad.

// capitulo_02 · analisis_comparativo

2. Análisis Comparativo v1.0 → v2.0

Para evitar duplicaciones y asegurar que las funcionalidades marcadas como "(Nuevo)" en v2.0 efectivamente no existían en el SRS v1.0, se realizó un barrido módulo por módulo sobre los 75 requisitos funcionales del baseline. La tabla 2.1 sintetiza el resultado de ese barrido, identificando para cada módulo afectado: (a) el alcance original en v1.0, (b) el delta introducido en v2.0, y (c) el veredicto de novedad (confirmado, parcial, o ya cubierto). Esta verificación es crítica porque el SRS v1.0 ya incluía elementos como el leaderboard semanal (RF-054 a RF-057) y el sistema de misiones (RF-059), que podrían confundirse con las nuevas reglas de XP y logros.

2.1 Tabla de Brecha (Gap Analysis)

Módulo	Cobertura v1.0	Delta v2.0 (Nuevo)	Veredicto
Landing Page	No existía módulo dedicado. La home pública no estaba formalmente especificada; sólo se mencionaba "app instalable" como futuro alcance.	Módulo 2 nuevo completo: terminal interactiva, contadores dinámicos globales con lectura pre-agregada.	NUEVO
Perfil y Estadísticas	RF-009 a RF-018 cubren onboarding, edición básica (nombre, intereses, nivel, presentación), avatar y avatar por defecto. No había panel de estadísticas filtrable ni vista previa.	Vista previa en vivo (RF-PROF-02) y estado vacío en Mis Estadísticas con CTA (RF-PROF-05).	NUEVO
Gamificación	RF-058 a RF-062 ya definen XP por acciones, misiones, logros desbloqueables y panel de progreso. El leaderboard (RF-054-057) ya incluye reset semanal y bonus a top 3.	Reglas de XP visibles en UI del leaderboard (RF-GAM-03) y elevación de logros a entidad de primera clase con CRUD (RF-GAM-04).	PARCIAL
Recursos	RF-036 a RF-043 cubren publicación, tipos, filtros, descargas, popularidad, favoritos y edición. No había límite explícito de adjuntos ni validación pre-publicación.	Límite estricto de 3 adjuntos con validación pre-publicación (RF-RES-02).	NUEVO
Comunidad / Directos	RF-044 a RF-048 especifican calendario, reserva de cupos, lista de espera, recordatorios y cancelación. No había estado vacío definido.	Estado vacío "// PRÓXIMOS" cuando no hay directos programados (RF-COM-03).	NUEVO

Tabla 2.1 — Análisis comparativo por módulo entre v1.0 y v2.0.

2.2 Lectura del Análisis

El análisis revela tres hallazgos importantes para el equipo de producto. **Primero**, el módulo de Landing Page es completamente nuevo y representa la primera superficie pública no autenticada de la plataforma: hasta v1.0, AutoDev no tenía especificación formal de página de aterrizaje, lo que sugiere que el producto

evolucionó desde una aplicación puramente autenticada hacia una estrategia de captación exterior. Esto tiene implicaciones de costo en Firestore porque los contadores deben servirse a usuarios no autenticados sin consumir cuota de lectura por visita.

Segundo, la gamificación es el módulo con mayor solapamiento: v1.0 ya contenía logros desbloqueables (RF-061) y un panel de progreso (RF-062), pero v2.0 eleva los logros a entidad de primera clase con CRUD administrativo, modelo de datos propio y visualización destacada en el perfil. Esto no es un RF nuevo desde cero, sino una evolución de un RF existente —el equipo debe decidir si amplía el contrato de RF-061 o si crea RF-GAM-04 como superconjunto y deprecia parcialmente al anterior. La recomendación técnica es la segunda opción, manteniendo RF-061 como caso especial del nuevo RF-GAM-04 durante un ciclo de deprecación.

Tercero, los cambios en perfil, recursos y comunidad son aditivos puros y de bajo esfuerzo (≤ 3 SP cada uno), por lo que pueden entregarse en un mismo sprint sin riesgo arquitectónico. El RF-RES-02 (límite de 3 adjuntos) merece atención particular: aunque su complejidad funcional es baja, requiere una Cloud Function de validación pre-escritura en Firestore para que el límite se aplique incluso si un cliente malicioso omite la validación del frontend. Esta Cloud Function debe usar el patrón de *Firestore Security Rules + pre-write trigger* ya establecido en v1.0 para otros campos validados.

// capitulo_03 · nuevas_funcionalidades

3. Nuevas Funcionalidades Detalladas

Cada una de las 8 funcionalidades nuevas se documenta a continuación con una estructura uniforme: identificador del requisito, descripción funcional, justificación de negocio, impacto técnico en la arquitectura Firebase, dependencias con otros RF y consideraciones de seguridad. Esta sección es la referencia canónica para el equipo de desarrollo durante la implementación del sprint v2.0.

3.1 RF-LAND-01 — Landing Page con Terminal Interactiva

// modulo_02 · landing_page · prioridad_must

Descripción. El sistema debe mostrar una página de aterrizaje pública (no autenticada) con una terminal interactiva que emule el flujo de comandos `$ autodev init` y `$ ver cursos --gratis`, junto con contadores dinámicos globales que reflejen el número total de developers registrados, posts publicados, comentarios realizados, cursos disponibles y recursos compartidos en la plataforma.

Justificación de negocio. La landing page es el primer punto de contacto para captación orgánica (SEO, redes sociales, enlaces externos). La terminal interactiva refuerza el posicionamiento de AutoDev como "aprende a automatizar como un developer de verdad", diferenciándolo de plataformas de cursos genéricas. Los contadores dinámicos transmiten prueba social en tiempo real y aumentan la tasa de conversión visitante→registro según benchmarks de comunidades técnicas similares (Dev.to, Hashnode).

Impacto técnico Firebase. Nuevo documento `app_metadata/counters` en Firestore con campos `developersCount`, `postsCount`, `commentsCount`, `coursesCount`, `resourcesCount`. Las Cloud Functions existentes `onPostCreate`, `onCommentCreate` y `onUserCreate` ya actualizan contadores desnormalizados en `/users/{uid}` — deben extenderse con una transacción que también incremente el campo correspondiente en `app_metadata/counters`. La terminal interactiva se implementa en el cliente (Next.js + React) sin dependencias backend adicionales: usa una secuencia pre-grabada de comandos con efecto máquina de escribir.

Dependencias. Ninguna hacia otros RF nuevos. Reutiliza el patrón de contadores desnormalizados ya probado en v1.0 (RF-041 popularidad de recursos, RF-052 estadísticas globales de miembros). **Seguridad:** las reglas de Firestore deben permitir lectura pública (*allow read: if true;*) exclusivamente sobre el documento `app_metadata/counters`; el resto de colecciones siguen requiriendo autenticación.

3.2 RF-LAND-02 — Contadores en Tiempo Real sin Comprometer Cuota

// modulo_02 · landing_page · prioridad_must · optimizacion_cuota

Descripción. Los contadores mostrados en la landing page deben reflejar los totales casi en tiempo real (latencia < 5 segundos desde el evento que los modifica) sin comprometer la cuota de lectura de la base de datos. La implementación debe realizar exactamente **una lectura de un documento pre-agregado** por visita a la landing, sin consultas de agregación (SUM, COUNT) sobre colecciones.

Justificación de negocio. Firestore cobra por lectura y escritura de documento. Una landing viral con 100.000 visitas/día generaría 100.000 lecturas/día si se hace una lectura simple del documento `counters`, lo

cual es aceptable (costo ~\$0.04 USD/día a \$0.36 por 100K lecturas). En cambio, una consulta de agregación tipo *SELECT COUNT(*) FROM users* escanearía todos los documentos y multiplicaría el costo por varios órdenes de magnitud. Esta optimización es el factor crítico que hace viable la landing pública desde el punto de vista de costo unitario por visitante.

Impacto técnico Firebase. El documento *app_metadata/counters* definido en RF-LAND-01 es la única fuente de verdad. Para evitar lecturas adicionales por renderizado, el cliente debe cachear el valor en *sessionStorage* con TTL de 60 segundos y opcionalmente suscribirse a *onSnapshot* del documento solo si el usuario permanece en la landing más de 30 segundos (lazy realtime). Las Cloud Functions que actualizan contadores deben usar *FieldValue.increment(1)* dentro de una transacción para garantizar atomicidad incluso bajo concurrencia alta (picos de tráfico tras una mención en redes).

Consideraciones de consistencia. Dado que Firestore es eventualmente consistente entre escrituras y lecturas de agregados, los contadores pueden mostrar un desfase de 1-3 segundos respecto al evento real. Este desfase es aceptable para la landing y debe documentarse en el SLA interno. Para escenarios críticos donde se necesite consistencia fuerte (ej: panel de administración), se debe leer el contador directamente desde la fuente desnormalizada en lugar de la caché del cliente.

3.3 RF-PROF-02 — Vista Previa en Tiempo Real del Perfil

// modulo_03 · perfil · prioridad_should · UX

Descripción. La edición del perfil debe incluir una vista previa lateral en tiempo real que refleje los cambios de Foto de Perfil, Nombre Completo y Biografía a medida que el usuario los edita, antes de pulsar "Guardar cambios". La vista previa debe usar el mismo componente visual que el perfil público para garantizar consistencia WYSIWYG.

Justificación de negocio. La vista previa reduce el ciclo de feedback de "editar → guardar → ver perfil → editar de nuevo" a "editar → ver en vivo → guardar". Esto disminuye errores de tipeo, imágenes mal recortadas y biografías con formatos rotos, mejorando la calidad percibida de los perfiles públicos. Es una mejora estándar en plataformas sociales (LinkedIn, GitHub) y su ausencia se percibe como falta de pulido.

Impacto técnico Firebase. Cambio puramente client-side: el componente *EditProfileForm* mantiene estado local con los campos editables, y un componente hermano *ProfilePreview* recibe los mismos valores como props. No requiere nuevas Cloud Functions ni cambios en Firestore. La subida de foto de perfil debe usar *Resumable Upload* de Cloud Storage con preview local vía *URL.createObjectURL()* antes de confirmar la subida —si el usuario cancela, el archivo nunca se sube y se evita el costo de escritura en Storage.

Dependencias. Reutiliza RF-PROF-01 (edición básica de perfil). **Accesibilidad:** la vista previa debe tener *aria-live="polite"* para que los usuarios con lectores de pantalla reciban anuncios de los cambios sin interrumpir la edición.

3.4 RF-PROF-05 — Estado Vacío en Mis Estadísticas

// modulo_03 · perfil · prioridad_should · empty_state

Descripción. Si el usuario no tiene actividad registrada en "Mis Estadísticas" (cero posts, cero comentarios, cero vistas), el sistema debe mostrar un estado vacío con el texto *// sin datos aún* y un Call to Action destacado con el botón *" + Crear Post "* que redirige al editor de posts.

Justificación de negocio. Los estados vacíos son uno de los puntos de mayor fricción en onboarding emocional: un usuario nuevo que llega a un dashboard vacío sin instrucciones tiene una probabilidad de abandono (churn) significativamente mayor. Un CTA claro aumenta la activación D1 (usuarios que publican su primer post en el primer día) que es la métrica predictiva más fuerte de retención D30 según el benchmark de v1.0.

Impacto técnico Firebase. El cliente consulta los documentos */users/{uid}/stats/overview* y */users/{uid}/stats/posts_ranking* (desnormalizados por Cloud Functions ya existentes en v1.0). Si ambos documentos no existen o tienen todos los campos en cero, se renderiza el componente *EmptyState* con un ícono, el texto *// sin datos aún* y el botón. La consulta no agrega costo adicional: ya era necesaria para llenar el dashboard cuando hay datos.

Consideraciones de diseño. El texto *// sin datos aún* sigue la convención visual de terminal definida en la landing page (RF-LAND-01), reforzando la identidad de marca "developer-first". El botón *+ Crear Post* debe usar el color de acento (#2d7ab3) y disparar el evento analytics *empty_state_cta_clicked* para medir la efectividad del patrón.

3.5 RF-GAM-03 — Reglas de XP y Recompensas Explícitas en el Leaderboard

// modulo_04 · gamificacion · prioridad_must · transparencia

Descripción. El Leaderboard Semanal debe mostrar explícitamente, en una sección visible del componente, las "Reglas de XP" (cómo ganar: Post +10, Tarea +15, Comentario +5, etc.) y las "Recompensas" (Top 1 = +100 XP, Top 2 = +50 XP, Top 3 = +25 XP, por ejemplo). Actualmente v1.0 (RF-054 a RF-057) ya calcula el leaderboard y otorga bonus a top 3 (500/300/200 XP), pero no expone estas reglas al usuario final en la UI.

Justificación de negocio. La transparencia en las reglas de gamificación aumenta la motivación intrínseca (Autonomy y Mastery en el framework SDT de Ryan & Deci). Cuando los usuarios entienden exactamente qué acciones generan XP y qué recompensas obtienen, su participación en posts y comentarios se incrementa. Es un patrón exitoso documentado en Stack Overflow (reputation badges), Duolingo (XP leagues) y Khan Academy (energy points).

Impacto técnico Firebase. Las reglas de XP ya existen como constantes en el código de las Cloud Functions (*XP_RULES* y *WEEKLY_REWARDS*). El cambio consiste en externalizarlas a un documento Firestore *app_metadata/gamification_config* con la estructura `{ postXP: 10, commentXP: 5, taskXP: 15, weeklyRewards: { top1: 100, top2: 50, top3: 25 } }`. El cliente lee este documento una vez al cargar el leaderboard y renderiza la sección. Esto permite al equipo de producto ajustar las reglas sin desplegar código nuevo en las Cloud Functions.

Dependencias y seguridad. Depende de RF-054 (cálculo del leaderboard) y RF-057 (premios bonus) ya existentes. **Importante:** las Cloud Functions deben seguir usando las constantes compiladas en código para validar otorgamiento de XP, no leer el documento *gamification_config* en tiempo de ejecución —esto evitaría que un atacante con permiso de escritura (imposible por reglas, pero defensa en profundidad) modifique las reglas para auto-asignarse XP. El documento es solo fuente de visualización para el cliente, no fuente de verdad de ejecución.

3.6 RF-GAM-04 — Sistema de Logros (Achievements) de Primera Clase

// modulo_04 · gamificacion · prioridad_must · modelo_datos_nuevo

Descripción. El sistema debe otorgar Logros (Achievements) por hitos específicos —ejemplos: primer post, 10 comentarios, Top 10 semanal, 30 días activo— y mostrarlos destacados en el perfil del usuario. v1.0 ya menciona "logros desbloqueables por hitos" en RF-061, pero v2.0 los eleva a entidad de primera clase con modelo de datos propio, CRUD administrativo, plantillas visuales (icono, título, descripción, rareza) y una Cloud Function reactiva que los evalúa automáticamente cuando se cumplen las condiciones.

Justificación de negocio. Los logros son una de las palancas más efectivas de retención a medio plazo (D7-D30). Mientras que la XP motiva acciones individuales, los logros crean objetivos a medio plazo que el usuario persigue durante varias sesiones. El sistema de rareza (común, raro, épico, legendario) introduce un elemento de colección que fomenta la revisión periódica del perfil y la comparación social con otros miembros.

Impacto técnico Firebase. Nueva colección */achievements* (definiciones globales) y subcolección */users/{uid}/achievements* (logros desbloqueados por usuario). Esquema de definición: `{ id, code, title,`

description, iconUrl, rarity, criteria: { type, threshold } }. Esquema de desbloqueo: *{ achievementId, unlockedAt, contextSnapshot }*. La Cloud Function *evaluateAchievements* se invoca después de cada evento que pueda desbloquear un logro (post creado, comentario creado, like recibido, ranking actualizado) y verifica en una transacción cuáles logros aplican.

Defensa contra duplicados. La transacción debe usar *transaction.get()* sobre el documento de logro del usuario; si ya existe, se aborta silenciosamente. Para evitar N+1 lecturas, las definiciones de logros se cachean en memoria de la Cloud Function (con TTL de 5 minutos) usando el patrón ya establecido para *gamification_config*. **Estimación de costo:** con 30 logros definidos y 619 usuarios activos, el worst case es ~18K lecturas/día sobre la colección de definiciones —aceptable y mitigable con la caché.

3.7 RF-RES-02 — Límite Estricto de 3 Adjuntos por Recurso

// modulo_05 · recursos · prioridad_must · validacion_seguridad

Descripción. El sistema debe limitar estrictamente a **3** los archivos adjuntos permitidos por recurso, validando este límite antes de la publicación. v1.0 (RF-036 a RF-043) permite adjuntos pero no especifica un límite máximo, lo que abre la posibilidad de abuso: un usuario malintencionado podría adjuntar decenas de archivos por recurso, saturando Cloud Storage y degradando el rendimiento del listado de recursos.

Justificación de negocio. El límite de 3 adjuntos alinea con la convención visual del formulario (3 botones de archivos adjuntos visibles en las capturas de pantalla) y con el patrón mental de "recurso conciso" que la plataforma quiere promover. Recursos con 5+ adjuntos típicamente indican que el autor debería fragmentar el contenido en varios recursos especializados. El límite también acota el costo de Storage y el ancho de banda de descarga.

Impacto técnico Firebase. Validación en dos capas: **(1) Frontend**, el componente *ResourceForm* deshabilita el botón "Agregar adjunto" cuando ya hay 3 archivos en el array local, mostrando un tooltip "Límite máximo: 3 archivos". **(2) Backend**, la Cloud Function *onResourceCreate* (extensión de la existente en v1.0) valida que el array *attachments* tenga *length* ≤ 3 antes de escribir en */resources/{id}*; si excede, rechaza con error *permission-denied* y no cobra la escritura. Adicionalmente, la regla de Firestore *allow create: if request.resource.data.attachments.size() ≤ 3* aplica como tercera capa de defensa.

Validación de tipo y tamaño. Aprovechando este cambio, se debe estandarizar también el tipo MIME permitido (pdf, zip, tar.gz, json, md, ipynb) y el tamaño máximo por archivo (10 MB, alineado con el límite de Storage y por debajo del límite de 1 MB de Firestore para metadata). Estos valores deben externalizarse a *app_metadata/resources_config* para permitir ajustes sin redeploy.

3.8 RF-COM-03 — Estado Vacío en Sección Directos

// modulo_06 · comunidad · prioridad_should · empty_state

Descripción. La sección "Directos" debe renderizar un estado vacío con el texto *"// PRÓXIMOS"* y un mensaje explicativo del tipo *"próximamente anunciaremos nuevas sesiones en vivo sobre automatización, n8n, WhatsApp e IA"* cuando no haya eventos programados en el calendario. Actualmente v1.0 (RF-044) asume que siempre hay al menos una sesión futura en la colección */streams*, lo que produce un bug visual si la colección está vacía.

Justificación de negocio. El estado vacío comunica expectativa y mantiene al usuario informado en lugar de mostrar un loader infinito o un grid vacío. Cuando el equipo de contenido programa un nuevo directo, los usuarios que ya vieron el estado vacío son más propensos a revisar la sección nuevamente. El texto *// PRÓXIMOS* refuerza la identidad terminal de la marca ya establecida en RF-LAND-01 y RF-PROF-05.

Impacto técnico Firebase. El cliente consulta */streams* con filtro *where("scheduledAt", ">", now())* y *limit(10)*. Si el snapshot retorna un array vacío, se renderiza el componente *EmptyStreams* en lugar del grid. Esta consulta ya existía en v1.0 para llenar el calendario; el cambio es solo agregar la rama de *empty state* en el renderizado condicional. Adicionalmente, se debe suscribir a *onSnapshot* para que cuando el admin cree un nuevo directo, el estado vacío desaparezca automáticamente sin necesidad de recargar la página.

RF-COM-04 — Directos gratuitos con cupo limitado (refuerzo). Aunque v1.0 ya cubre cupos en RF-045 y RF-046, v2.0 refuerza el mensaje visual: la sección debe indicar explícitamente "Miembros gratis, cupo limitado" cerca del botón de reserva. Este texto es copy estático, no requiere cambios en el modelo de datos.

// capitulo_04 · criterios_aceptacion · bdd_gherkin

4. Criterios de Aceptación y Escenarios BDD

Cada funcionalidad nueva se acompaña de al menos un escenario BDD en formato Gherkin (Given/When/Then). Estos escenarios son la base para la automatización de pruebas con Cucumber.js + Playwright y deben ser revisados por QA antes de marcar cualquier RF como "Done". Se mantiene la convención de v1.0: cada escenario nombra documentos Firestore específicos para que el fixture de prueba pueda sembrarlos sin ambigüedad.

4.1 Escenarios — Landing Page (RF-LAND-01, RF-LAND-02)

```
Feature: Landing Page con contadores dinámicos
  Como visitante no autenticado
  Quiero ver la landing page con contadores actualizados
  Para percibir la vitalidad de la comunidad AutoDev

  Scenario: Visualización de contadores dinámicos en la Landing
    Given la base de datos tiene 619 usuarios y 180 posts registrados
    And el documento "app_metadata/counters" contiene:
      | developersCount | 619 |
      | postsCount | 180 |
      | commentsCount | 286 |
      | coursesCount | 12 |
      | resourcesCount | 15 |
    When un usuario no autenticado visita la Landing Page
    Then el componente muestra "619 developers" y "180 posts"
    And los números se renderizan con animación de conteo ascendente
    And la terminal interactiva muestra la secuencia "$ autodev init ->"

  Scenario: Contador servido con una sola lectura Firestore
    Given el documento "app_metadata/counters" existe y está actualizado
    When el cliente renderiza la Landing Page por primera vez
    Then se realiza exactamente 1 lectura de documento Firestore
    And no se ejecuta ninguna consulta de agregación (COUNT/SUM)
    And el valor se cachea en sessionStorage con TTL de 60 segundos

  Scenario: Actualización casi en tiempo real tras nuevo post
    Given la Landing Page está abierta con snapshot activo
    When un usuario autor publica un nuevo post
    Then la Cloud Function "onPostCreate" incrementa "postsCount" en 1
    And el snapshot del cliente recibe la actualización en menos de 5 segundos
    And el contador visual transiciona de "180" a "181"
```

4.2 Escenarios — Perfil y Estadísticas (RF-PROF-02, RF-PROF-05)

```
Feature: Vista previa en tiempo real del perfil
  Como usuario editando mi perfil
  Quiero ver una vista previa en vivo de mis cambios
  Para evitar guardar cambios que se vean mal

  Scenario: Vista previa refleja cambios de biografía en vivo
    Given el usuario "yvan" está en la página "/perfil/editar"
```

```
And su biografía actual es "Developer en formación"
When el usuario escribe " - Especialista en n8n" en el campo biografía
Then el panel de vista previa muestra:
"Developer en formación - Especialista en n8n"
And el botón "Guardar cambios" permanece habilitado
And no se ha realizado ninguna escritura en Firestore
```

```
Scenario: Vista previa de foto de perfil sin subir el archivo
Given el usuario está editando su perfil sin foto previa
When el usuario selecciona "avatar.png" en el input de foto
Then la vista previa muestra la imagen usando URL.createObjectURL()
And el archivo NO se ha subido a Cloud Storage todavía
When el usuario pulsa "Cancelar"
Then el archivo se descarta sin costo de Storage
```

Feature: Estado vacío en Mis Estadísticas

```
Como usuario nuevo sin actividad
Quiero ver un mensaje claro y un CTA
Para entender qué debo hacer para empezar
```

```
Scenario: Usuario nuevo sin posts ni comentarios ve estado vacío
Given el usuario "maria" se registró hoy
And el documento "/users/maria/stats/overview" no existe
When el usuario navega a "/perfil/mis-estadisticas"
Then se muestra el componente EmptyState con texto "// sin datos aún"
And se muestra un botón "+ Crear Post" con color de acento #2d7ab3
And no se muestra ninguna tarjeta de métrica
```

```
Scenario: Usuario con actividad ve dashboard normal
Given el usuario "yvan" tiene 1 post y 0 comentarios
And su documento "/users/yvan/stats/overview" tiene:
| engagement | 12 |
| views | 47 |
| saves | 3 |
| activeDays | 2 |
When el usuario navega a "/perfil/mis-estadisticas"
Then se muestran 4 tarjetas de métrica con los valores indicados
And NO se muestra el componente EmptyState
```


4.3 Escenarios — Gamificación (RF-GAM-03, RF-GAM-04)

Feature: Reglas de XP explícitas en el Leaderboard

Como miembro de la comunidad

Quiero ver las reglas de XP y las recompensas

Para entender cómo subir en el ranking semanal

Scenario: Visualización de reglas y recompensas en el Leaderboard

Given el Leaderboard Semanal está renderizado para "06 Jul → 12 Jul, 2026"

And el documento "app_metadata/gamification_config" contiene:

| postXP | 10 |

| commentXP | 5 |

| taskXP | 15 |

| weeklyRewards | {top1:100, top2:50, top3:25} |

When el usuario abre el Leaderboard

Then se muestra una sección "Reglas de XP" con las entradas:

| Post +10 XP | Tarea +15 XP | Comentario +5 XP |

And se muestra una sección "Recompensas" con:

| Top 1 = +100 XP | Top 2 = +50 XP | Top 3 = +25 XP |

And el botón "¡sube al top!" está visible

Feature: Sistema de Logros (Achievements)

Como usuario activo

Quiero desbloquear logros por hitos

Para sentir progresión a medio plazo

Scenario: Desbloqueo de logro "Primer Post"

Given el usuario "lucia" no tiene logros desbloqueados

And la colección "/achievements" contiene el logro:

| code | FIRST_POST |

| title | Primer Post |

| rarity | common |

| criteria | { type: "post_count", threshold: 1 } |

When "lucia" publica su primer post

Then la Cloud Function "evaluateAchievements" se ejecuta

And verifica que el post_count de "lucia" ahora es 1

And crea el documento "/users/lucia/achievements/FIRST_POST"

And el campo "unlockedAt" tiene timestamp actual

And se envía una notificación in-app "¡Logro desbloqueado: Primer Post!"

Scenario: No se duplica el logro si ya está desbloqueado

Given el usuario "lucia" ya tiene el logro "FIRST_POST"

When "lucia" publica un segundo post

Then la Cloud Function verifica "FIRST_POST" ya existe en la transacción

And aborta silenciosamente sin crear duplicado

And no se envía notificación

Scenario: Logro "Top 10 semanal" se evalúa tras reset del leaderboard

Given el leaderboard de la semana anterior finalizó con "lucia" en posición 7

And existe el logro:

| code | TOP_10_WEEKLY |

| rarity | rare |

| criteria | { type: "weekly_rank", threshold: 10 } |

When se ejecuta la Cloud Function "resetWeeklyRanking"

Then se evalúa el logro "TOP_10_WEEKLY" para todos los top 10

And se crea el documento "/users/lucia/achievements/TOP_10_WEEKLY"

4.4 Escenarios — Recursos y Comunidad (RF-RES-02, RF-COM-03)

Feature: Límite de 3 adjuntos por recurso
Como autor publicando un recurso
Quiero que el sistema limite los adjuntos a 3
Para mantener los recursos concisos y controlar costos de Storage

Scenario: Frontend bloquea el 4to adjunto
Given el usuario está en el formulario "Publicar Recurso"
And ya ha agregado 3 archivos adjuntos
When el usuario intenta agregar un 4to archivo
Then el botón "Agregar adjunto" está deshabilitado
And se muestra tooltip "Límite máximo: 3 archivos"

Scenario: Backend rechaza recurso con 4 adjuntos
Given un cliente malicioso omite la validación del frontend
And envía un payload con 4 archivos adjuntos a la Cloud Function
When la Cloud Function "onResourceCreate" procesa el evento
Then la transacción verifica `attachments.length == 4`
And rechaza con error "permission-denied"
And no se escribe en `/resources`
And no se suben archivos a Cloud Storage

Scenario: Regla de Firestore como tercera capa de defensa
Given un atacante con credenciales válidas intenta escribir directamente
When ejecuta `set()` en `/resources/{newId}` con 5 attachments
Then la regla de Firestore `request.resource.data.attachments.size() <= 3`
And rechaza la escritura con "PERMISSION_DENIED"

Feature: Estado vacío en sección Directos
Como miembro de la comunidad
Quiero ver un mensaje claro cuando no hay directos programados
Para saber que volveré más tarde

Scenario: Sin directos programados muestra estado vacío
Given la colección `/streams` no tiene documentos con `scheduledAt > now()`
When el usuario navega a `/directos`
Then se renderiza el componente `EmptyStreams`
And se muestra el texto `// PRÓXIMOS`
And se muestra "próximamente anunciaremos nuevas sesiones en vivo"
And se muestra "Miembros gratis, cupo limitado"

Scenario: Estado vacío desaparece cuando se crea un directo
Given el usuario está viendo el `EmptyStreams` con snapshot activo
When el admin crea un documento en `/streams` con `scheduledAt` futuro
Then el snapshot recibe el nuevo documento en menos de 3 segundos
And el `EmptyStreams` desaparece
And se renderiza el grid con el nuevo directo

// capitulo_05 · impacto_arquitectura · firebase

5. Impacto en la Arquitectura Firebase

Este capítulo consolida todos los cambios necesarios en el ecosistema Firebase para soportar las 8 funcionalidades nuevas. Se organiza en cuatro secciones: cambios en el modelo de datos de Firestore, nuevas Cloud Functions y modificaciones a las existentes, ajustes a las reglas de seguridad, y cambios en Cloud Storage. Cada cambio se acompaña de la justificación de costo y la métrica de cuota esperada con 619 usuarios activos (baseline actual).

5.1 Cambios en el Modelo de Datos Firestore

Se introducen dos nuevas colecciones y se extienden dos colecciones existentes. La colección `/app_metadata` ya existe en v1.0 con documentos como `app_metadata/counters` (introducido formalmente en RF-LAND-01 pero ya usado informalmente para popularidad de recursos); se le añaden dos documentos nuevos: `gamification_config` y `resources_config`. La colección `/achievements` es completamente nueva y contiene las definiciones globales de logros. La subcolección `/users/{uid}/achievements` almacena los desbloques por usuario.

Ruta	Tipo	RF origen	Esquema / Campos
<code>app_metadata/counters</code>	Existente+ext	RF-LAND-01	developersCount, postsCount, commentsCount, coursesCount, resourcesCount (numéricos)
<code>app_metadata/gamification_config</code>	Nuevo	RF-GAM-03	postXP, commentXP, taskXP, weeklyRewards: {top1, top2, top3}
<code>app_metadata/resources_config</code>	Nuevo	RF-RES-02	maxAttachments: 3, allowedMimeTypes: [...], maxFileSizeMB: 10
<code>achievements/{achievementId}</code>	Nuevo	RF-GAM-04	code, title, description, iconUrl, rarity (common rare epic legendary), criteria: {type, threshold}
<code>users/{uid}/achievements/{code}</code>	Nuevo	RF-GAM-04	achievementId, unlockedAt, contextSnapshot
<code>users/{uid}/stats/overview</code>	Existente	RF-PROF-05	Sin cambios; usado para detectar EmptyState
<code>streams/{streamId}</code>	Existente	RF-COM-03	Sin cambios; cliente maneja snapshot vacío

Tabla 5.1 — Cambios en el modelo de datos Firestore.

5.2 Cloud Functions — Nuevas y Modificadas

Las Cloud Functions son el motor reactivo de AutoDev. v1.0 define un patrón consistente: triggers `onCreate` sobre colecciones principales que actualizan contadores desnormalizados y disparan notificaciones. v2.0 extiende este patrón sin introducir nuevos paradigmas. La tabla 5.2 lista las 4 Cloud Functions afectadas, indicando si son nuevas o modificaciones a las existentes, junto con el costo estimado de invocaciones por día.

Función	Tipo	Trigger	Cambio	Inv/día
<code>onUserCreate</code>	Modificar	<code>auth.user().onCreate</code>	Añadir transaction: incrementa <code>developersCount</code> en <code>app_metadata/counters</code>	~10
<code>onPostCreate</code>	Modificar	<code>firestore /posts onCreate</code>	Incrementa <code>postsCount</code> ; invoca <code>evaluateAchievements(FIRST_POST, etc.)</code>	~50
<code>onCommentCreate</code>	Modificar	<code>firestore /comments onCreate</code>	Incrementa <code>commentsCount</code> ; invoca <code>evaluateAchievements(COMMENTER_10, etc.)</code>	~80
<code>evaluateAchievements</code>	Nueva	Llamada por <code>onPostCreate/onCommentCreate/resetWeeklyRanking</code>	Transacción: lee <code>/users/{uid}/achievements</code> , verifica criterios, crea desbloques	~140

Tabla 5.2 — Cloud Functions nuevas y modificadas.

Configuración de instancias. Las funciones modificadas mantienen `minInstances=1` para latencia baja, siguiendo la convención de v1.0 para funciones críticas. La nueva función `evaluateAchievements` se invoca desde otras funciones ya cálidas, por lo que hereda la instancia caliente y no necesita su propio `minInstances`. Estimación de costo adicional mensual: ~\$2 USD por invocaciones + ~\$5 USD por memoria de instancia cálida adicional, dentro del tier gratuito+ de Firebase Blaze plan.

5.3 Reglas de Seguridad Firestore

Las reglas de seguridad deben actualizarse para reflejar los nuevos documentos públicos y las nuevas colecciones. Los cambios críticos son: (1) lectura pública explícita sobre `app_metadata/counters` y `app_metadata/gamification_config`, (2) validación del límite de adjuntos en `/resources`, y (3) reglas de propiedad sobre `/users/{uid}/achievements` (solo el usuario puede leer los suyos; solo Cloud Functions pueden escribir).

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // — Lectura pública para Landing Page (RF-LAND-01, RF-LAND-02, RF-GAM-03) —
    match /app_metadata/{doc} {
      allow read: if true;
      allow write: if false; // solo Cloud Functions con Admin SDK
    }

    // — Recursos: límite de 3 adjuntos (RF-RES-02) —
    match /resources/{resourceId} {
      allow read: if true;
      allow create: if request.auth != null
        && request.resource.data.attachments.size() <= 3
        && request.resource.data.attachments.size() >= 0;
      allow update, delete: if request.auth != null
        && resource.data.authorId == request.auth.uid;
    }

    // — Logros: definiciones públicas, desbloques privados (RF-GAM-04) —
    match /achievements/{achievementId} {
```

```
allow read: if true;
allow write: if false; // solo Admin SDK desde Cloud Functions
}
match /users/{uid}/achievements/{code} {
allow read: if request.auth != null && request.auth.uid == uid;
allow write: if false; // solo evaluateAchievements (Admin SDK)
}

// ... resto de reglas de v1.0 sin cambios
}
}
```

5.4 Cloud Storage — Cambios y Límites

Cloud Storage recibe cambios menores pero importantes. RF-RES-02 introduce validación explícita de tipo MIME y tamaño máximo por archivo. RF-PROF-02 cambia el flujo de subida de avatar para usar preview local antes de confirmar. Las reglas de Storage deben alinearse con estos límites para defensa en profundidad, no confiando únicamente en la validación del cliente.

Para el límite de tamaño, se recomienda usar *Resumable Upload* con *maxBytes* configurado a 10 MB en el cliente y validar el tamaño en la Cloud Function post-subida. Si el archivo excede, se elimina inmediatamente con *bucket.file().delete()* y se notifica al usuario. Esta doble validación previene que un cliente modificado suba archivos grandes que persistan en Storage consumiendo cuota sin ser asociados a ningún recurso.

5.5 Estimación de Cuota y Costo Mensual

La tabla 5.3 sintetiza el costo incremental mensual esperado por las nuevas funcionalidades, asumiendo el baseline actual de 619 usuarios activos, 180 posts acumulados y ~50 posts/día nuevos. Todos los valores están en USD y asumen el plan Firebase Blaze. Los costos incluyen lecturas, escrituras, eliminaciones y ancho de banda, pero excluyen FCM (sin cambios) y Analytics (sin cambios).

Funcionalidad	Lecturas/día	Escrituras/día	Costo/mes (USD)
RF-LAND-01/02 — Landing Page	~1,000	~50	\$0.50
RF-PROF-02 — Vista previa perfil	0 (client-side)	0	\$0.00
RF-PROF-05 — Empty state stats	~600	0	\$0.20
RF-GAM-03 — Reglas XP visibles	~600	0	\$0.20
RF-GAM-04 — Sistema de logros	~600	~140	\$1.20
RF-RES-02 — Límite 3 adjuntos	0	0	\$0.00 (ahorro por evitar abuso)
RF-COM-03 — Empty state directos	~200	0	\$0.10
Total incremental	~4,000	~190	~\$2.20

Tabla 5.3 — Estimación de cuota y costo mensual incremental.

// capitulo_06 · trazabilidad · moscow · roadmap

6. Matriz de Trazabilidad y Priorización MoSCoW

La matriz de trazabilidad vincula cada RF nuevo con su escenario BDD, el componente UI afectado, el módulo backend y el responsable sugerido. La priorización MoSCoW (Must / Should / Could / Won't) guía el orden de implementación del sprint v2.0. Must equivale a bloqueante para release (no se puede publicar v2.0 sin estos); Should es importante pero posponible a v2.0.1 si es necesario; Could se incluye si hay tiempo; Won't se descarta explícitamente para esta versión.

6.1 Matriz de Trazabilidad

RF ID	Escenario BDD	Componente UI	Backend / Firestore
RF-LAND-01	Visualización de contadores dinámicos	LandingPage, Terminal, CounterBadge	app_metadata/counters (lectura)
RF-LAND-02	Contador con 1 lectura; actualización realtime	LandingPage (useCounters hook)	onSnapshot app_metadata/counters + sessionStorage
RF-PROF-02	Vista previa refleja biografía en vivo	EditProfileForm, ProfilePreview	Ninguno (client-side puro)
RF-PROF-05	Usuario nuevo ve EmptyState; con actividad ve dashboard	MyStatsDashboard, EmptyState	Lectura /users/{uid}/stats/overview
RF-GAM-03	Visualización de reglas y recompensas	Leaderboard, XPRulesPanel, RewardsPanel	app_metadata/gamification_config (lectura)
RF-GAM-04	Desbloqueo "Primer Post"; no duplicado; Top 10 semanal	AchievementsGrid, AchievementBadge, ProfileAchievements	/achievements, /users/{uid}/achievements, evaluateAchievements CF
RF-RES-02	Frontend bloquea 4to adjunto; backend y reglas rechazan	ResourceForm, AttachmentInput	onResourceCreate validación + regla Firestore
RF-COM-03	Sin directos → EmptyState; snapshot elimina vacío	DirectosList, EmptyStreams	Lectura /streams where scheduledAt > now()

Tabla 6.1 — Matriz de trazabilidad RF ↔ BDD ↔ UI ↔ Backend.

6.2 Priorización MoSCoW y Plan de Sprint

La priorización MoSCoW se basa en tres criterios: (1) bloqueo de release —si la funcionalidad es esencial para la promesa de v2.0; (2) costo de implementación —si el beneficio justifica el esfuerzo; (3) riesgo técnico —si posponer introduce deuda técnica significativa. El siguiente cuadro sintetiza el resultado y propone un plan de sprint de 2 semanas con 3 desarrolladores full-stack y 1 ingeniero Firebase.

Categoría	RF incluidos	Razón	Sprint
MUST	RF-LAND-01, RF-LAND-02, RF-GAM-03, RF-GAM-04, RF-RES-02	Bloqueantes para release v2.0. Sin ellos, la landing no existe, el leaderboard pierde transparencia clave, y el abuso de adjuntos queda sin mitigar.	Sprint 1 (sem 1)

Categoría	RF incluidos	Razón	Sprint
SHOULD	RF-PROF-02, RF-PROF-05, RF-COM-03	Mejoras UX importantes pero no bloqueantes. Posponibles a v2.0.1 si hay presión de tiempo.	Sprint 2 (sem 2)
COULD	Refuerzo visual RF-COM-04 ("Miembros gratis, cupo limitado")	Copy estático, se incluye si hay holgura en Sprint 2.	Sprint 2 (si hay tiempo)
WON'T	Sistema de rareza visual avanzada (animaciones, holográficos)	Pospuesto a v2.1. La rareza se almacena pero no se anima en esta versión.	v2.1

Tabla 6.2 — Priorización MoSCoW de funcionalidades nuevas.

6.3 Plan de Implementación y Definición de Hecho (DoD)

El plan de implementación se divide en dos sprints de una semana cada uno. **Sprint 1** se enfoca en los Must: el ingeniero Firebase trabaja en paralelo sobre las Cloud Functions y reglas de seguridad, mientras dos desarrolladores full-stack construyen la Landing Page y el sistema de logros. El tercer desarrollador full-stack lidera RF-RES-02 y RF-GAM-03 que son los de menor complejidad técnica. **Sprint 2** aborda los Should con foco en pulido UX y validación de escenarios BDD con automatización Playwright.

Definición de Hecho (DoD) común a todos los RF: (1) Escenarios BDD en Gherkin aprobados por QA; (2) Cobertura de pruebas unitarias $\geq 80\%$ en código nuevo; (3) Pruebas E2E en Playwright pasando en CI; (4) Reglas de Firestore actualizadas y probadas con emulador; (5) Documentación de Cloud Functions en `/docs/functions.md` actualizada; (6) Métricas de costo monitoreadas en Firebase Console durante 24 horas en staging; (7) Code review aprobado por al menos un par; (8) Migración de datos (si aplica) probada en staging y documentada con rollback plan.

Criterios de salida del sprint: todos los RF Must con DoD completo, todos los RF Should con DoD completo o pospuestos explícitamente a v2.0.1 con ticket creado, smoke test E2E pasando en producción después del despliegue canary al 10% del tráfico durante 24 horas sin errores P0/P1, y métricas de cuota Firestore dentro del $\pm 20\%$ de la estimación de la tabla 5.3.

6.4 Riesgos Técnicos y Mitigaciones

Se identifican tres riesgos técnicos principales que el equipo debe monitorear durante la implementación.

Riesgo 1 — Picos de tráfico en Landing Page: una mención viral puede multiplicar el tráfico por 100. Mitigación: caché en CDN (Firebase Hosting + Cloudflare) con TTL de 60 segundos sobre la landing, y caché en sessionStorage del cliente. **Riesgo 2 — Doble otorgamiento de logros:** si la Cloud Function `evaluateAchievements` se ejecuta dos veces (retry de Firestore trigger), podría crear duplicados. Mitigación: usar `transaction.get()` con idempotencia basada en `achievementId + userId` como clave primaria del documento. **Riesgo 3 — Bypass del límite de 3 adjuntos:** un cliente modificado podría intentar subir 5 archivos. Mitigación: triple capa (frontend + Cloud Function + regla Firestore) ya descrita en RF-RES-02.